

Practical Lab Report

Part 3: Simulated Malware Behavior (Safe Environment)

Field	Details
Student Name	Joseph Osemudiamhen Okoekoh
Student ID	CSC/22/01/0047
Course	COMP404 Computer Security
Lecturer	Dr. George K. Agordzo
Lab Environment	Kali Linux (kali@kali, 192.168.239.128) - single VM, analysis and target machine
Date	23 May 2026

1. Introduction

This report documents a safe, self-contained exercise simulating basic malware behavior on the Kali Linux workstation. A harmless shell script (`harmless_simulation.sh`) was created to mimic three common malicious behaviors in a controlled way: creating hidden files and a log entry, writing a harmless test file, and establishing a scheduled (cron) task to simulate persistence. The script was then executed, its effects were detected and analyzed using standard Linux tools (`ls`, `cat`, `crontab`, `top`, `journalctl`), and finally all traces were removed and the cleanup was verified. No real malicious code, network activity, or system damage was involved at any point.

2. Part A: Script Creation and Execution

The simulation script was created using `nano` and made executable, then run from the home directory:

```
nano harmless_simulation.sh
chmod +x harmless_simulation.sh
./harmless_simulation.sh
```

The script performs the following actions: it creates a hidden directory `~/.sys_update`, writes a timestamped entry to `~/.sys_update/log.txt` to simulate an access log, creates a harmless test file on the Desktop (`test_file.txt`), and adds a cron job that periodically appends to `~/.sys_update/cron_log.txt` to simulate a persistence mechanism. Running the script produced the expected output:

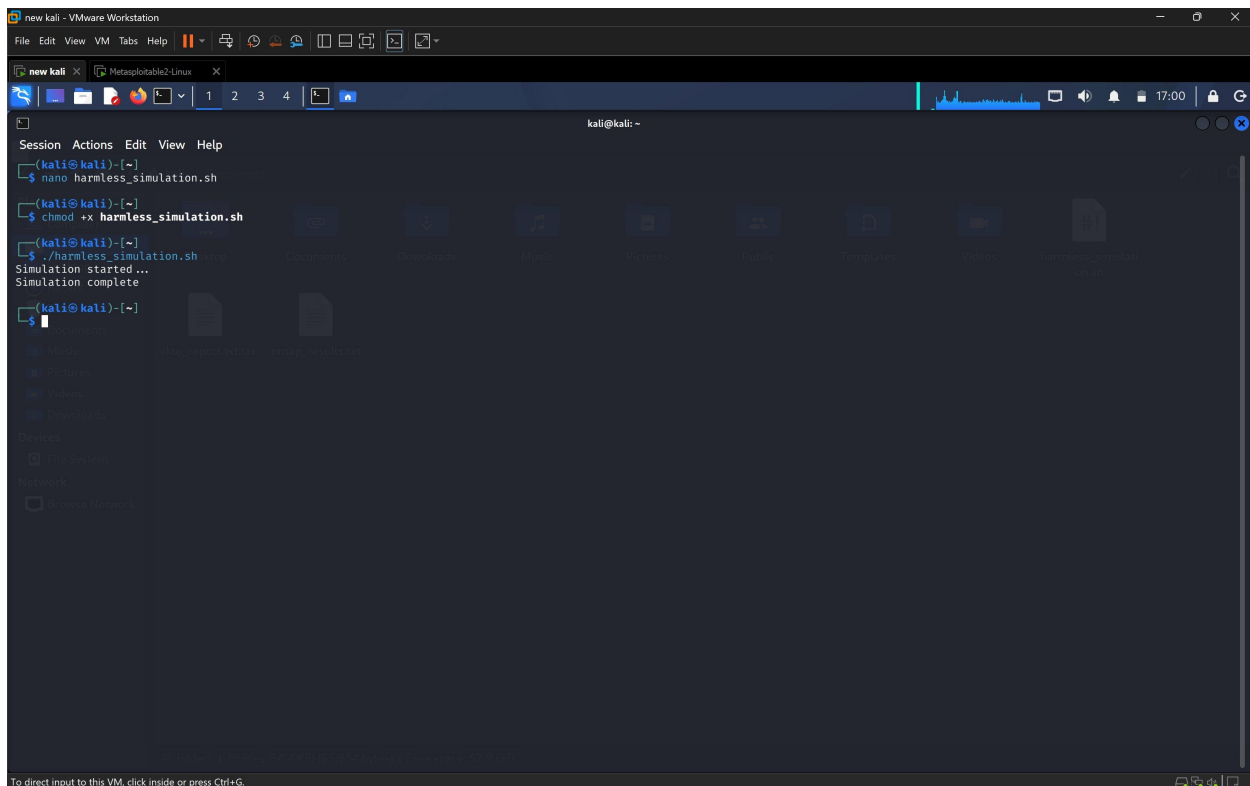
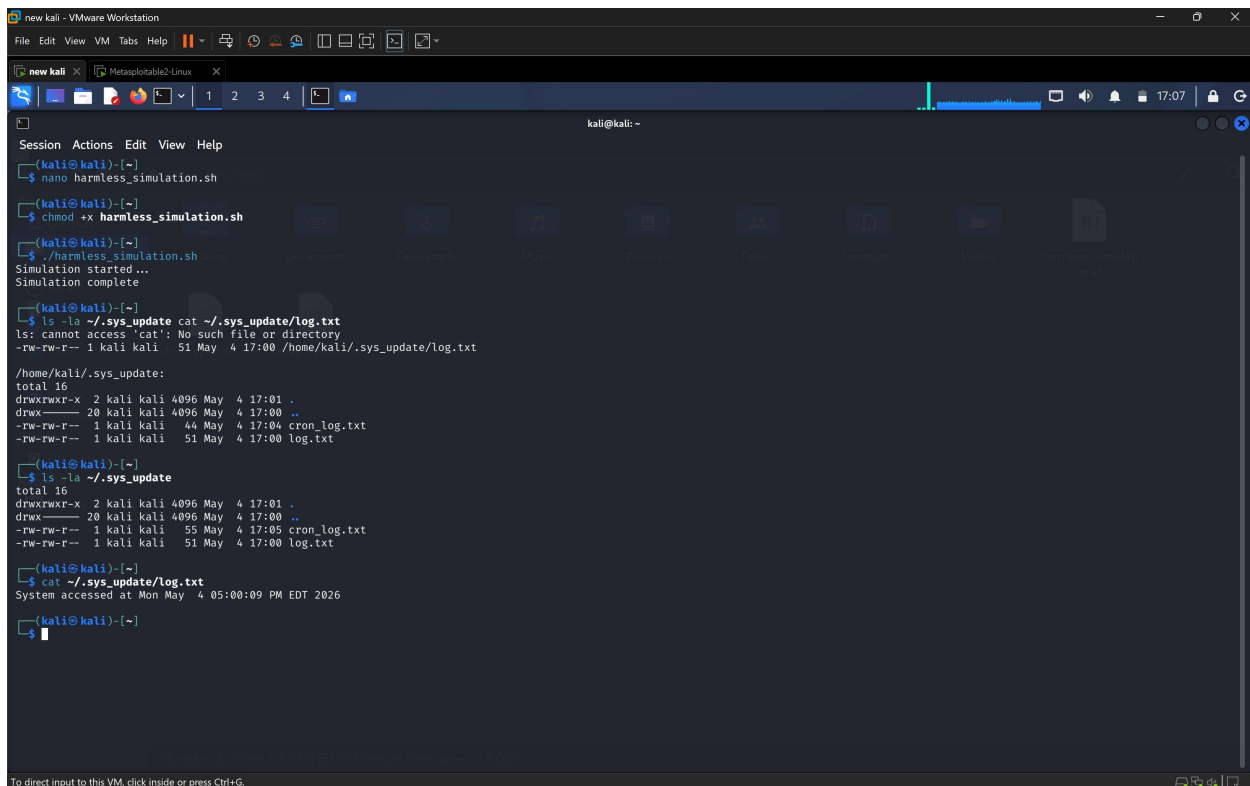


Figure 1: Creating, making executable, and running `harmless_simulation.sh` - output shows 'Simulation started...' and 'Simulation complete'.

3. Part B: Detection and Analysis

3.1 Evidence of Created Files

The hidden directory `~/.sys_update` was inspected to confirm the files created by the script. It contains `log.txt` (the simulated access log) and `cron_log.txt` (created/updated once the cron job began running). The contents of `log.txt` show a timestamped entry confirming the script's file-creation behavior:



```
new kali - VMware Workstation
File Edit View VM Tabs Help
new kali X Metasploitable2-Linux X
kali@kali: ~
Session Actions Edit View Help
(kali@kali)~$ nano harmless_simulation.sh
(kali@kali)~$ chmod +x harmless_simulation.sh
(kali@kali)~$ ./harmless_simulation.sh
Simulation started...
Simulation complete
(kali@kali)~$ ls -la ~/.sys_update cat ~/.sys_update/log.txt
ls: cannot access 'cat': No such file or directory
-rw-rw-r-- 1 kali kali 51 May 4 17:00 /home/kali/.sys_update/log.txt

/home/kali/.sys_update:
total 16
drwxrwxr-x 2 kali kali 4096 May 4 17:01 .
drwx----- 20 kali kali 4096 May 4 17:00 ..
-rw-rw-r-- 1 kali kali 44 May 4 17:04 cron_log.txt
-rw-rw-r-- 1 kali kali 51 May 4 17:00 log.txt
(kali@kali)~$ ls -la ~/.sys_update
total 16
drwxrwxr-x 2 kali kali 4096 May 4 17:01 .
drwx----- 20 kali kali 4096 May 4 17:00 ..
-rw-rw-r-- 1 kali kali 55 May 4 17:05 cron_log.txt
-rw-rw-r-- 1 kali kali 51 May 4 17:00 log.txt
(kali@kali)~$ cat ~/.sys_update/log.txt
System accessed at Mon May 4 05:00:09 PM EDT 2026
(kali@kali)~$
```

Figure 2: `ls -la ~/.sys_update` and `cat ~/.sys_update/log.txt` - showing the hidden directory, `log.txt`, `cron_log.txt`, and the logged timestamp ('System accessed at Mon May 4 05:00:09 PM EDT 2026').

3.2 Cron Job (Persistence)

The `crontab -l` command was used to list scheduled tasks for the `kali` user, confirming that the script had installed a persistence-style cron job that runs every minute and appends a status line to `~/.sys_update/cron_log.txt`:

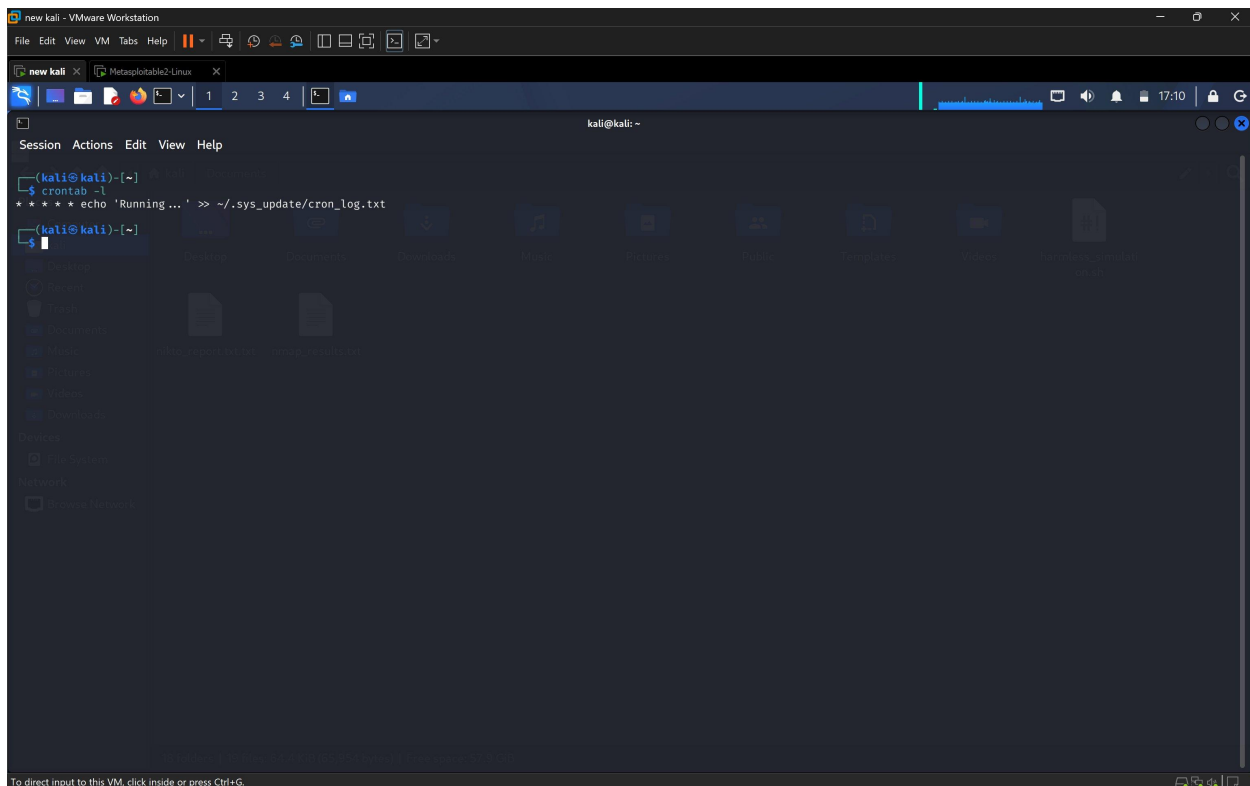


Figure 3: crontab -l showing the scheduled job ' * * * * echo "Running..." >> ~/.sys_update/cron_log.txt'.*

This mirrors how real malware often establishes persistence - by registering a recurring scheduled task so that it continues to run (or 'check in') even after a user logs out or the system is rebooted.

3.3 Process Monitoring

The top command was used to review running processes and overall system resource usage at the time of the simulation. This step represents the kind of routine process monitoring an analyst would perform to look for unfamiliar or resource-heavy processes that might indicate malicious activity:

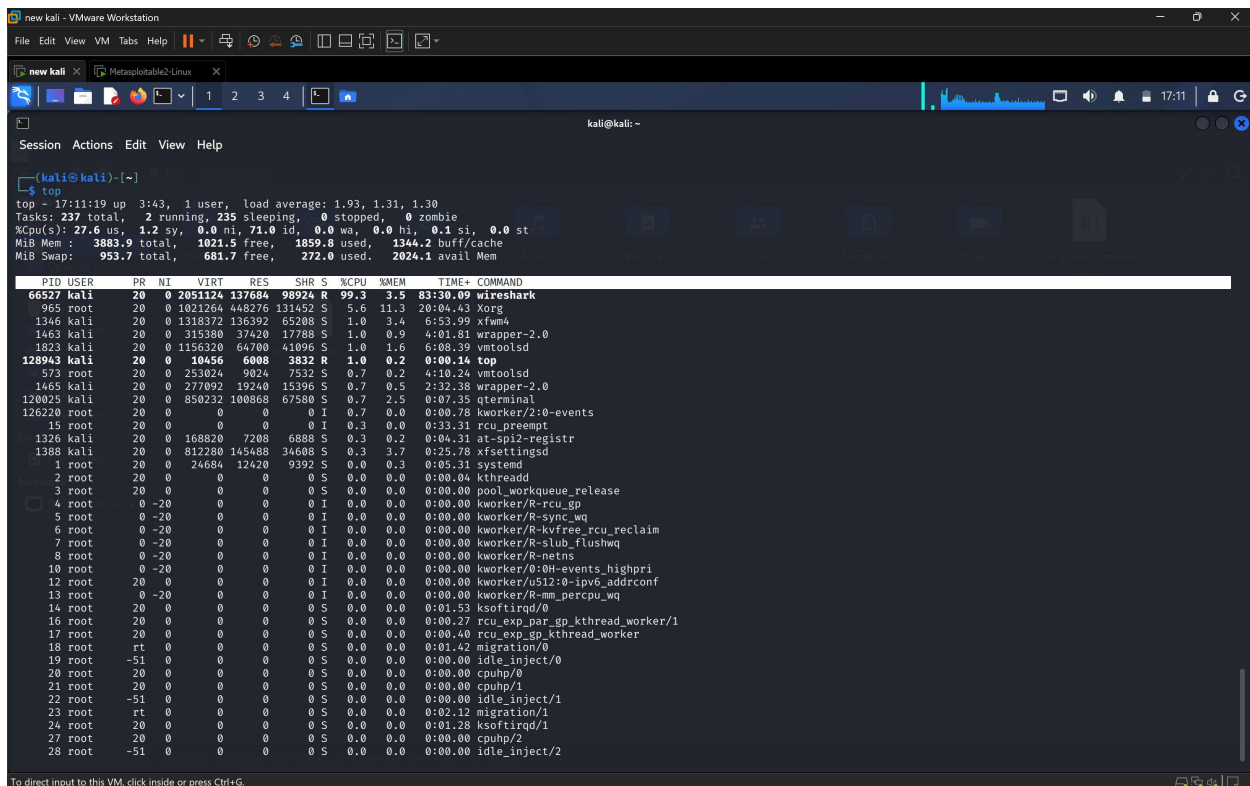
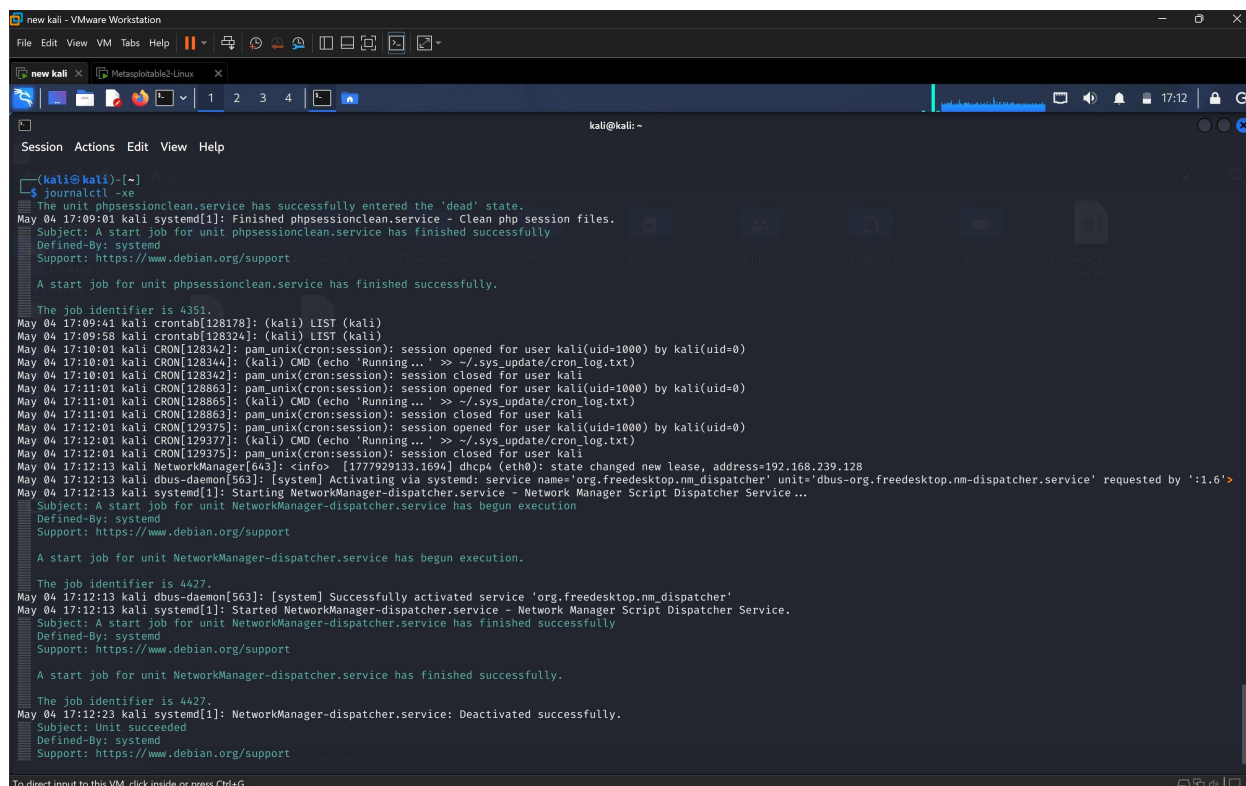


Figure 4: top output showing the active process list and system load at the time of the simulation.

In this case, the process list shows only expected, legitimate processes (desktop environment components, Wireshark, system services), with nothing unusual that would indicate the simulated script is still actively running as a foreground process - consistent with it having already completed and only its cron job remaining active in the background.

3.4 System Log Review

The journalctl -xe command was used to review recent system log entries. The logs clearly show the cron daemon executing the scheduled job once per minute, opening and closing a PAM session for the kali user each time, and appending to cron_log.txt - directly corresponding to the persistence mechanism installed by the script:



```
(kali@kali)~$ journalctl -xe
The unit phpsessionclean.service has successfully entered the 'dead' state.
May 04 17:09:01 kali systemd[1]: Finished phpsessionclean.service - Clean php session files.
Subject: A start job for unit phpsessionclean.service has finished successfully
Defined-By: systemd
Support: https://www.debian.org/support

A start job for unit phpsessionclean.service has finished successfully.

The job identifier is 4351.
May 04 17:09:41 kali crontab[128178]: (kali) LIST (kali)
May 04 17:09:58 kali crontab[128324]: (kali) LIST (kali)
May 04 17:10:01 kali CRON[128342]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
May 04 17:10:01 kali CRON[128344]: (kali) CMD (echo 'Running...' >> ~/.sys_update/cron_log.txt)
May 04 17:10:01 kali CRON[128342]: pam_unix(cron:session): session closed for user kali
May 04 17:11:01 kali CRON[128863]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
May 04 17:11:01 kali CRON[128865]: (kali) CMD (echo 'Running...' >> ~/.sys_update/cron_log.txt)
May 04 17:11:01 kali CRON[128863]: pam_unix(cron:session): session closed for user kali
May 04 17:12:01 kali CRON[129375]: pam_unix(cron:session): session opened for user kali(uid=1000) by kali(uid=0)
May 04 17:12:01 kali CRON[129377]: (kali) CMD (echo 'Running...' >> ~/.sys_update/cron_log.txt)
May 04 17:12:01 kali CRON[129375]: pam_unix(cron:session): session closed for user kali
May 04 17:12:13 kali NetworkManager[643]: <info> [1777929133.1694] dhcp4 (eth0): state changed new lease, address=192.168.239.128
May 04 17:12:13 kali dbus-daemon[563]: [system] Activating via systemd: service name='org.freedesktop.nm_dispatcher' unit='dbus-org.freedesktop.nm_dispatcher.service' requested by ':1.6'
May 04 17:12:13 kali systemd[1]: Started NetworkManager-dispatcher.service - Network Manager Script Dispatcher Service...
Subject: A start job for unit NetworkManager-dispatcher.service has begun execution
Defined-By: systemd
Support: https://www.debian.org/support

A start job for unit NetworkManager-dispatcher.service has begun execution.

The job identifier is 4427.
May 04 17:12:13 kali dbus-daemon[563]: [system] Successfully activated service 'org.freedesktop.nm_dispatcher'
May 04 17:12:13 kali systemd[1]: Started NetworkManager-dispatcher.service - Network Manager Script Dispatcher Service.
Subject: A start job for unit NetworkManager-dispatcher.service has finished successfully
Defined-By: systemd
Support: https://www.debian.org/support

A start job for unit NetworkManager-dispatcher.service has finished successfully.

The job identifier is 4427.
May 04 17:12:23 kali systemd[1]: NetworkManager-dispatcher.service: Deactivated successfully.
Subject: Unit succeeded
Defined-By: systemd
Support: https://www.debian.org/support
```

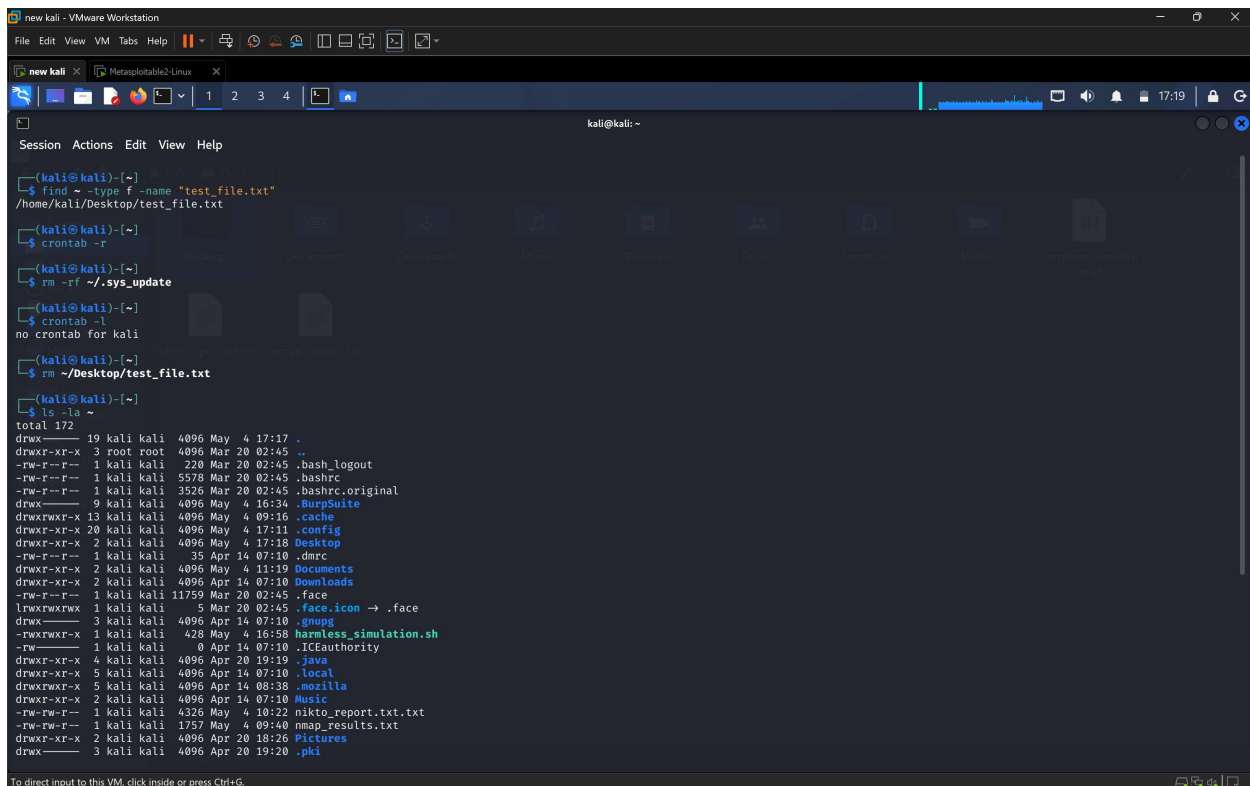
Figure 5: journalctl -xe showing repeated CRON entries executing 'echo 'Running...' >> ~/.sys_update/cron_log.txt' once per minute.

Discussion point: this kind of pattern - a short-lived process launched by cron at regular one-minute intervals, writing to a hidden directory - is exactly the kind of repeating, low-noise activity that can indicate malware persistence or a beaconing mechanism, and would be a key indicator for a defender reviewing system logs.

4. Part C: Defensive Actions (Removal and Cleanup)

Before removing the artifacts, the test file created on the Desktop was located using find to confirm its exact path. The persistence mechanism was then removed first (crontab -r), followed by deletion of the hidden directory and the test file:

```
find ~ -type f -name "test_file.txt"
crontab -r
rm -rf ~/.sys_update
rm ~/Desktop/test_file.txt
```

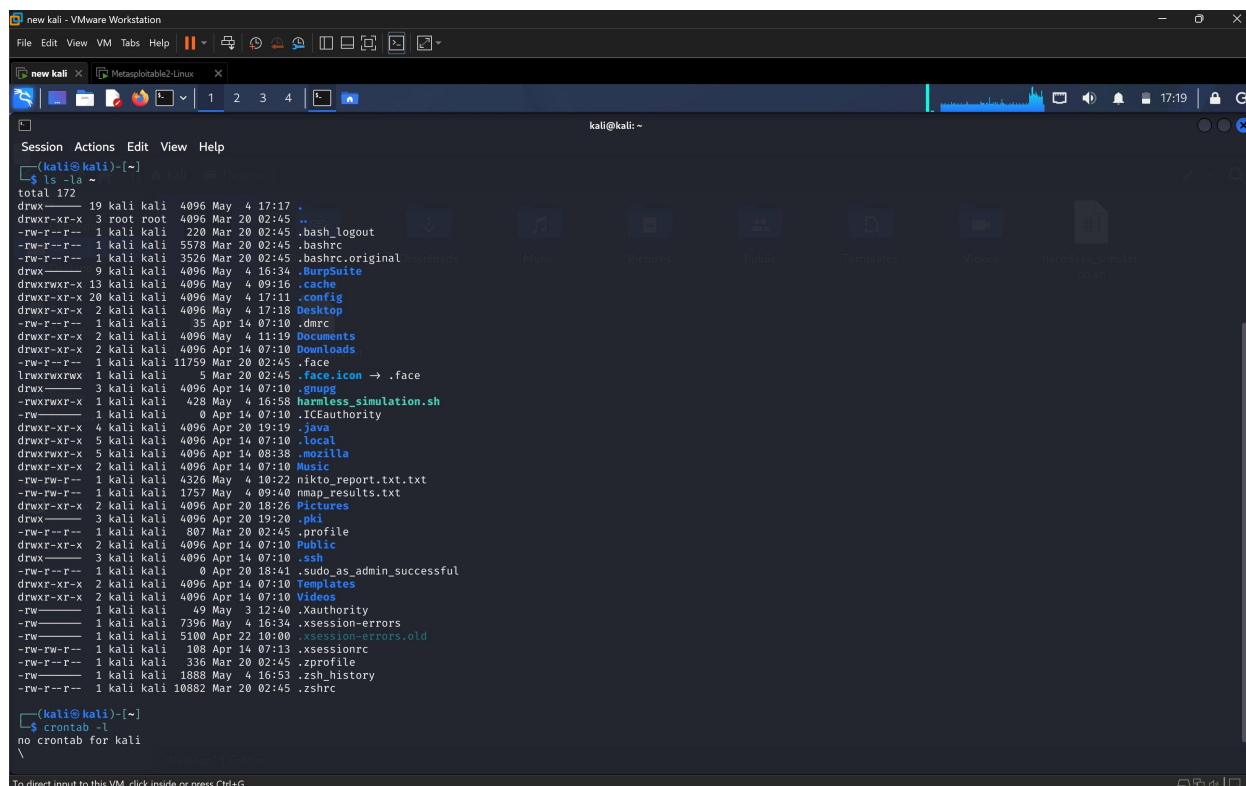


```
(kali@kali)~$ find -type f -name "test_file.txt"
/home/kali/Desktop/test_file.txt
(kali@kali)~$ crontab -r
(kali@kali)~$ rm -rf ~/.sys_update
(kali@kali)~$ crontab -l
no crontab for kali
(kali@kali)~$ rm ~/Desktop/test_file.txt
(kali@kali)~$ ls -la ~
total 172
drwxr-xr-x 19 kali kali 4096 May  4 17:17 .
drwxr-xr-x  3 root root 4096 Mar 20 02:45 ..
-rw-r--r--  1 kali kali  220 Mar 20 02:45 .bash_logout
-rw-r--r--  1 kali kali 5578 Mar 20 02:45 .bashrc
-rw-r--r--  1 kali kali 3526 Mar 20 02:45 .bashrc.original
drwxr-xr-x  9 kali kali 4096 May  4 16:34 .BursSuite
drwxrwxr-x 13 kali kali 4096 May  4 09:16 .cache
drwxr-xr-x 20 kali kali 4096 May  4 17:11 .config
drwxr-xr-x  2 kali kali 4096 May  4 17:18 Desktop
-rw-r--r--  1 kali kali  35 Apr 14 07:10 .dmrc
drwxr-xr-x  2 kali kali 4096 May  4 11:19 Documents
drwxr-xr-x  2 kali kali 4096 Apr 14 07:10 Downloads
-rw-r--r--  1 kali kali 11759 Mar 20 02:45 .face
lrwxrwxrwx  1 kali kali  5 Mar 20 02:45 .face.icon -> .face
drwxr-xr-x  3 kali kali 4096 Apr 14 07:10 .gnupg
-rwxrwxr-x  1 kali kali 428 May  4 16:58 harmless_simulation.sh
-rw-r--r--  1 kali kali  0 Apr 14 07:10 .ICFauthority
drwxr-xr-x  4 kali kali 4096 Apr 20 19:19 .java
drwxr-xr-x  5 kali kali 4096 Apr 14 07:10 .local
drwxrwxr-x  5 kali kali 4096 Apr 14 08:38 .mozilla
drwxr-xr-x  2 kali kali 4096 Apr 14 07:10 Music
-rw-rw-r--  1 kali kali 4326 May  4 10:22 nikto_report.txt.txt
-rw-rw-r--  1 kali kali 1757 May  4 09:40 nmap_results.txt
drwxr-xr-x  2 kali kali 4096 Apr 20 18:26 Pictures
drwxr-xr-x  3 kali kali 4096 Apr 20 19:20 .pki
```

Figure 6: Locating `test_file.txt` with `find`, then removing the cron job (`crontab -r`), the hidden `~/.sys_update` directory, and the Desktop test file.

4.1 Verification of Cleanup

Finally, `ls -la ~` and `crontab -l` were run to confirm that the cleanup was successful. The home directory listing no longer contains the `~/.sys_update` directory or the Desktop test file, and `crontab -l` reports 'no crontab for kali', confirming that the simulated persistence mechanism has been fully removed:



```
(kali@kali)~$ ls -la ~
total 172
drwxr-xr-x 19 kali kali 4096 May  4 17:17 .
drwxr-xr-x  3 root root 4096 Mar 20 02:45 ..
-rw-r--r--  1 kali kali 220 Mar 20 02:45 .bash_logout
-rw-r--r--  1 kali kali 5578 Mar 20 02:45 .bashrc
-rw-r--r--  1 kali kali 3526 Mar 20 02:45 .bashrc.original
drwxr-xr-x  9 kali kali 4096 May  4 16:34 .BurpSuite
drwxrwxr-x 13 kali kali 4096 May  4 09:16 .cache
drwxr-xr-x 20 kali kali 4096 May  4 17:11 .config
drwxr-xr-x  2 kali kali 4096 May  4 17:18 Desktop
-rw-r--r--  1 kali kali 35 Apr 14 07:10 .dmrc
drwxr-xr-x  2 kali kali 4096 May  4 11:19 Documents
drwxr-xr-x  2 kali kali 4096 Apr 14 07:10 Downloads
-rw-r--r--  1 kali kali 11759 Mar 20 02:45 .face
lrwxrwxrwx  1 kali kali 5 Mar 20 02:45 .face.icon -> .face
drwxr-xr-x  3 kali kali 4096 Apr 14 07:10 .gnupg
-rwxrwxr-x  1 kali kali 428 May  4 16:58 harmless_simulation.sh
-rw-r--r--  1 kali kali 0 Apr 14 07:10 .ICAuthority
drwxr-xr-x  4 kali kali 4096 Apr 20 19:19 .java
drwxr-xr-x  5 kali kali 4096 Apr 14 07:10 .local
drwxrwxr-x  5 kali kali 4096 Apr 14 08:38 .mozilla
drwxr-xr-x  2 kali kali 4096 Apr 14 07:10 Music
-rw-rw-r--  1 kali kali 4326 May  4 10:22 nikto_report.txt.txt
-rw-rw-r--  1 kali kali 1757 May  4 09:40 nmap_results.txt
drwxr-xr-x  2 kali kali 4096 Apr 20 18:26 Pictures
drwxr-xr-x  3 kali kali 4096 Apr 20 19:20 .pki
-rw-r--r--  1 kali kali 807 Mar 20 02:45 .profile
drwxr-xr-x  2 kali kali 4096 Apr 14 07:10 Public
drwxr-xr-x  3 kali kali 4096 Apr 14 07:10 .ssh
-rw-r--r--  1 kali kali 0 Apr 20 18:41 .sudo_as_admin_successful
drwxr-xr-x  2 kali kali 4096 Apr 14 07:10 Templates
drwxr-xr-x  2 kali kali 4096 Apr 14 07:10 Videos
-rw-r--r--  1 kali kali 49 May  3 12:40 .Xauthority
-rw-r--r--  1 kali kali 7396 May  4 16:34 .xsession-errors
-rw-r--r--  1 kali kali 5100 Apr 22 10:00 .xsession-errors.old
-rw-rw-r--  1 kali kali 108 Apr 14 07:13 .xsessionrc
-rw-r--r--  1 kali kali 336 Mar 20 02:45 .zprofile
-rw-r--r--  1 kali kali 1888 May  4 16:53 .zsh_history
-rw-r--r--  1 kali kali 10882 Mar 20 02:45 .zshrc

(kali@kali)~$ crontab -l
no crontab for kali
\
```

Figure 7: `ls -la ~` (no `.sys_update` directory present) and `crontab -l` reporting 'no crontab for kali', confirming successful cleanup. The `harmless_simulation.sh` script itself remains in the home directory.

Note: the `harmless_simulation.sh` script file itself is still present in the home directory, as only its effects (created files and the cron job) were targeted for removal in this exercise. For a complete cleanup it would also be removed with `rm ~/harmless_simulation.sh`.

5. Recommendations

- Regularly review user and system crontabs (`crontab -l` for each user, and `/etc/cron*`, `/etc/systemd/system/*.timer`) for unfamiliar scheduled jobs, especially ones that write to hidden directories or run at very short intervals.
- Monitor for the creation of hidden files and directories (names beginning with a dot) in user home directories, particularly ones created shortly after running an unfamiliar script or downloaded file.
- Use file integrity monitoring tools (e.g. AIDE, auditd) to alert on new or modified files in sensitive locations.
- Review system logs (`journalctl`, `/var/log/syslog`) periodically for repeating CRON entries or other recurring activity that does not correspond to known administrative tasks.
- When malware or a suspicious script is identified, remove its persistence mechanism first (cron jobs, systemd timers, startup scripts) before deleting its files, to prevent it from re-creating itself.
- After removing persistence and artifacts, always verify cleanup (e.g. `crontab -l`, `ls -la`, `find`) rather than assuming removal commands succeeded.

6. Conclusion

This exercise demonstrated, in a fully safe and contained way, the lifecycle of a simple persistence-based 'malware' simulation: creation of hidden files and a log entry, installation of a recurring cron job, detection of these artifacts through file listings, process monitoring, and system log review, and finally complete removal and verification of the persistence mechanism and associated files. The patterns observed - hidden directories, timestamped log files, and a once-per-minute cron job visible in journalctl - mirror real-world indicators of compromise, reinforcing the importance of routine monitoring of scheduled tasks, hidden files, and system logs as part of a defensive security posture.